

TW10 Application Testing 1

Links:

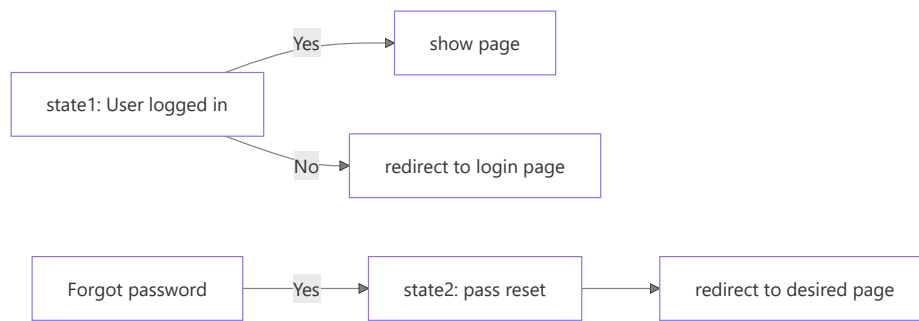
1. <https://flask-security-too.readthedocs.io/en/stable/configuration.html>
2. <https://dev.to/doabledanny/git-cheat-sheet-50-commands-free-pdf-and-poster-4gcn>

Levels of Testing

- Unit tests
 - *1 or part of controller* to add course to student → form working (dummy DB without entire system) → valid course id-student id → student ❌ >1 → payment gateway regression if 1st test fails →
 - Test individual units of functionality in isolation.
 - clear defined input & output
- integration tests
 - of set of units → combined system ✅ (❌ dependency violation) → version control systems
 - CI/CD** each commit to Main branch → re-evaluate *integration tests* multiple times a day but ❌ every change)
 - may work independently but compatibility issues on integration with other functions
- System-Testing
 - black-box** includes server, environment (google app engine/aws), DB, persistent connections → all aspects of behaviour (automatic **selenium** + manual)
- Non-functional tests
 - cost
 - No. of instances
 - performance under load
- User Acceptance Testing
 - deploy final system → limited **Beta** users → pre-production pilot → user Intelligent feedback
- Static Testing
 - Involves reviewing code **without executing** it *Code reviews, correctness proofs.*
- Dynamic Testing
 - Involves executing the application with different inputs. *Functional tests, performance tests*

Test Generation

- API-based
 - Swagger → OpenAPI** → generate tests for specific endpoints/scenarios → inject possible problem cases → Record API traffic → Data validation tests
- Abstract Tests
 - semi formal verbal description (❌ code)
 - apply to generic models
- Model-based testing
 - Scenarios: (Model generates test for each possible *states & transitions*)
 - abstract models → **executable tests**



- (G)UI testing
user interface
 - specific elements ☒ present on page
 - ☒ navigation links
 - what happens on random clicks on page ☒ corrupt DB, server crash
 - ☒ programmatic requests (captcha) ☒ browser
 - request gen: `capbara (ruby), py requests`
 - browser automate `selenium`
 - Security testing
 - generate invalid inputs to test app behaviour (`SQL injection`/`server overload` `DoS`)
 - `black-box` assume attacker doesn't know implement(miss edge cases) `white-box` know code implementation (over-complicated, focus on less IMP parts)
 - `White-box` designing test cases based on the **code detailed insights** but
 - `black-box` only focus on the **expected functionality** & behavior of the application, system testing, acceptance testing, security testing, etc.
 - **Fuzzing** random/semi-random inputs
 - `Grey-box`= Combination of `white-box` & `black-box`
 - **Regression Testing**: make sure previous tests does ☒ start failing (modification → break existing features) → `series of tests for all features`
- Authentication** (identify & verify like username, password)+ **Authorisation**(role-based access control rights to edit/view)

Coverage

Code coverage: Every line executed at least once.

Branch coverage: All decision branches tested.

Condition coverage: Each condition within decisions tested. (FTT, TFT, TTT, FFF, null etc)

Pytest

PYTHON

```
def func(x)
    return x**2
def test_ans():
    assert func(3)=5
def f():
    raise SystemExit(1)
def test*():
    with pytest.raises(SystemExit):
        f()
Fail: assert 0
```

- Running pytest without mentioning a filename will run all files of format **test_*.py** or ***_test.py** in the current directory and subdirectories. Pytest automatically identifies those files as test files, for other filenames by explicitly mentioning them.
- Pytest requires the test function names to start with test **test***
- To execute the tests containing a string in its name: **file if not then fn name**
- **Test Fixtures:** setup some data b4 test & remove after test (Pytest folder with dummy DB/users/files) [defined, reliable and consistent context for the tests.](#)

```

import os
import tempfile
import pytest
import flaskr # Added this import
from flaskr import create_app
from flaskr.db import init_db

@pytest.fixture
def client(): #provides a fresh Flask test client for every test,
which simulates browser-like behavior.
    db_fd, db_path = tempfile.mkstemp()
    app = create_app({'TESTING': True, 'DATABASE': db_path})

    with app.test_client() as client:
        with app.app_context():
            init_db()
        yield client
    os.close(db_fd)
    os.unlink(db_path)

def test_empty_db(client):
    response = client.get('/') #simulates a GET request.
    assert b'No entries here so far' in response.data

def login(client, username, password):
    return client.post('/login', data=dict(
        username=username,
        password=password)
        , follow_redirects=True) #simulates a POST request.

def logout(client):
    return client.get('/logout', follow_redirects=True)

def test_login_logout(client):
    username = flaskr.app.config["USERNAME"]
    password = flaskr.app.config["PASSWORD"]

    # Test successful login
    response = login(client, username, password) #is in bytes, so
compare using b'Text'.
    assert response.status_code == 200
    assert b'You were logged in' in response.data

```

```

# Test successful logout
response = logout(client) # Corrected function call
assert b'You were logged out' in response.data

# Test invalid username
response = login(client, f'{username}x', password) # Corrected
function call
assert b'Invalid username' in response.data

# Test invalid password
response = login(client, username, f'{password}x') # Corrected
function call
assert b'Invalid password' in response.data

@pytest.fixture
def setup_list():
    return ['Baskaran', 'Yashvi', 'Priya', 'Himanshu', 'Abhijeet',
            'Gunavanthi', 'Preethi', 'Kanwal']

```

```
pytest -k <substring>-v
```

SHELL

Markers are used to set various features/attributes to test functions. Pytest provides many inbuilt markers such as xfail, skip and parametrize.

```

@pytest.mark.<markername>
pytest -m <markername> -v #verbose

```

SHELL

Parameterization

Allows you to run the same test with different inputs.

```

import pytest

@pytest.mark.parametrize("a,b,result", [(1, 2, 3), (3, 4, 7), (5,
5, 10)])
def test_addition(a, b, result):
    assert a + b == result

```

PYTHON

Analogy: Like giving a quiz to multiple students with the same question but different answers.

Skipping & Expected Failures

PYTHON

```
import pytest

@pytest.mark.skip(reason="Work in progress")
def test_login():
    assert False

@pytest.mark.xfail(reason="Bug #101")
def test_feature():
    assert 1 == 2
```

- `@pytest.mark.skip`: Skip a test
- `@pytest.mark.xfail`: Expect it to fail (without breaking suite)

Grouping with Markers & Filtering

Markers allow grouping tests and selectively running them.

PYTHON

```
import pytest

@pytest.mark.slow
def test_large_data():
    assert True
```

Run only slow tests: `pytest -m slow`

Shared Fixtures with conftest.py

To avoid duplicating fixtures across test files.

Analogy: Like placing shared utensils in a kitchen drawer (`conftest.py`) instead of repeating them at every table.

- Structure

```
project/
├── tests/
│   ├── test_1.py
│   ├── test_2.py
│   └── conftest.py
```

- `conftest.py`

```
import pytest
@pytest.fixture
def fruit_basket():
    return ["apple", "banana", "cherry"]
```

- reuse fruit_basket

test_1.py:

```
def test_banana_in_basket(fruit_basket):
    assert "banana" in fruit_basket
```

- Pytest automatically looks for fixtures in **conftest.py**
- No need to import **fruit_basket** manually

Match the following types of testing with their functionality.



A. Regression testing	1. Beta Testing
B. User Acceptance testing	2. one step beyond integration testing, includes server and environment
C. System Testing	3. Simulates actual user interaction, allows to script browser
D. System testing Automation	4. Type of testing that runs after every change to ensure that the change introduces no unintended breaks.

Which of the following is the correct matching?

OPTIONS :

☐ A → 1, B → 2, C → 3, D → 4

☒ A → 4, B → 1, C → 2, D → 3

☐ A → 4, B → 3, C → 2, D → 1

☐ A → 3, B → 2, C → 1, D → 4

Consider the following function to be tested and test functions given in the Python code snippet below.

test_file.py

```
import pytest

def square(x):
    sum = 0
    for counter in range(x):
        sum += x
    return sum

@pytest.mark.marker1
def testcase_1():
    assert square(10) == 100

@pytest.mark.marker2
def testcase_2():
    assert square(4) == 4

@pytest.mark.marker3
def testcase_3():
    assert square(5) == 25

@pytest.mark.marker4
def testcase_4():
    assert square(6) == 6
```

On running this file on the terminal using pytest, the summary of the output is;

```
===== 1 failed, 3 deselected, 4 warnings in 0.04s =====
```

What command will result into the outcome given above?

Question 12 : 640653815509

[View Solutions \(0\)](#) Total

Match the following types of testing with their functionality.

A. White Box Testing	1. Beta Testing
B. User Acceptance testing	2. One step beyond integration Testing
C. System testing	3. Simulates actual user interaction, allows to script browser
D. System testing Automation	4. Considers internal functioning of the system

Which of the following is the correct matching?

OPTIONS :

☐ A → 1, B → 2, C → 3, D → 4

☐ A → 4, B → 3, C → 2, D → 1

☒ A → 4, B → 1, C → 2, D → 3

☐ A → 3, B → 2, C → 1, D → 4

[Discussions \(0\)](#)